
REQUIREMENTS NOT MET

- N/A

VIDEO FILE LINK

<https://youtube.com/shorts/CrQSTtrQYsw?si=2mAoPXElRjvJJvM>

PROBLEMS ENCOUNTERED

N/A

FUTURE WORK/APPLICATIONS

CPU's are the backbone of microprocessors, and in return computers, automation, and could power all sorts of modern electronics. We could make an automated math loop, create a loop for an LED display, etc.

PRE-LAB PART 1 EXERCISES

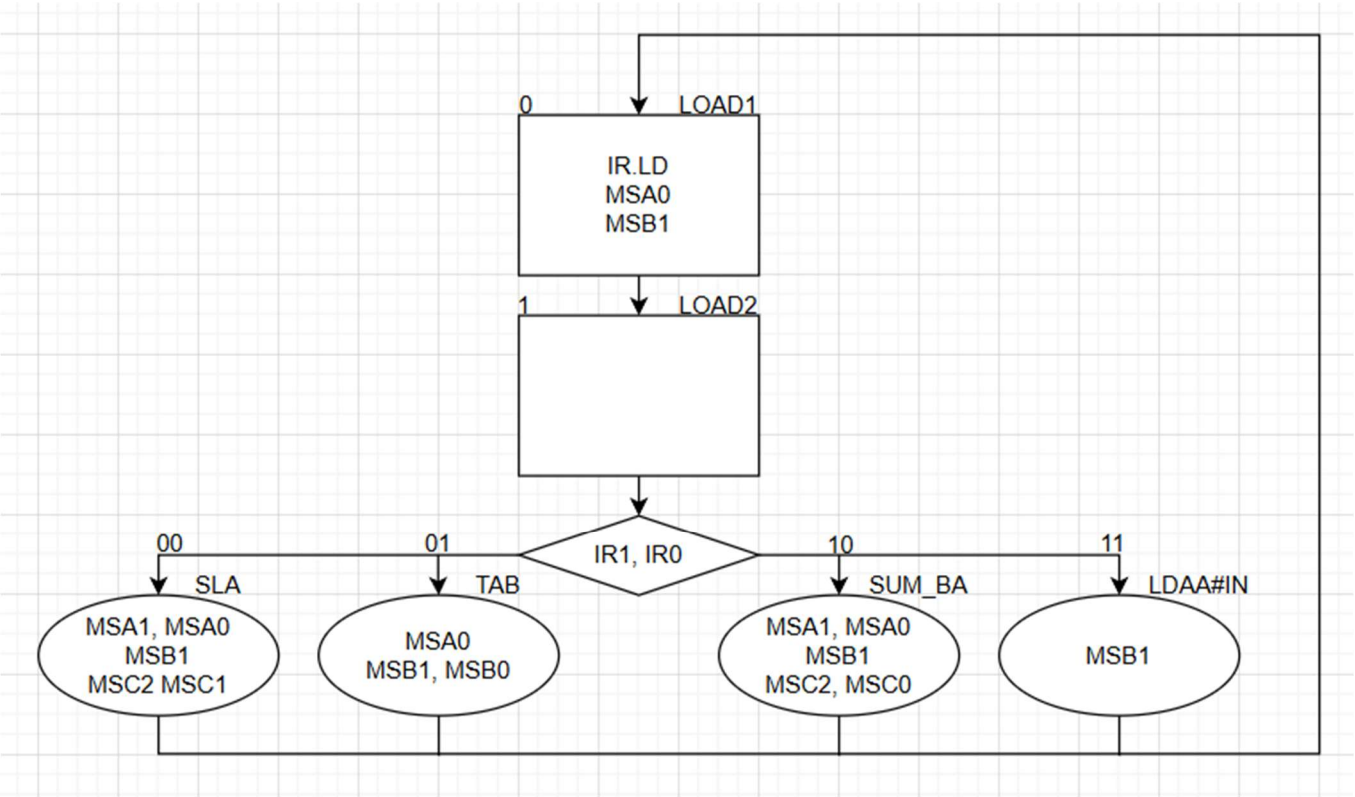


Figure 1: ASM for Lab 6 Part 1

Table 1: NSTT for Quartus CPU Design Part 1

IR[1]	IR[0]	Q	Q+	MSA[1]	MSA[0]	MSB[1]	MSB[0]	MSC[2]	MSC[1]	MSC[0]	IR.LD
-	-	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	1	0	1	1	0	0
0	1	1	0	0	1	1	1	0	0	0	0
1	0	1	0	1	1	1	0	1	0	1	0
1	1	1	0	0	0	1	0	0	0	0	0

Preliminary: Lab 4 reused and slightly modified to remove decoders from BSF (added a TON of messy outputs)

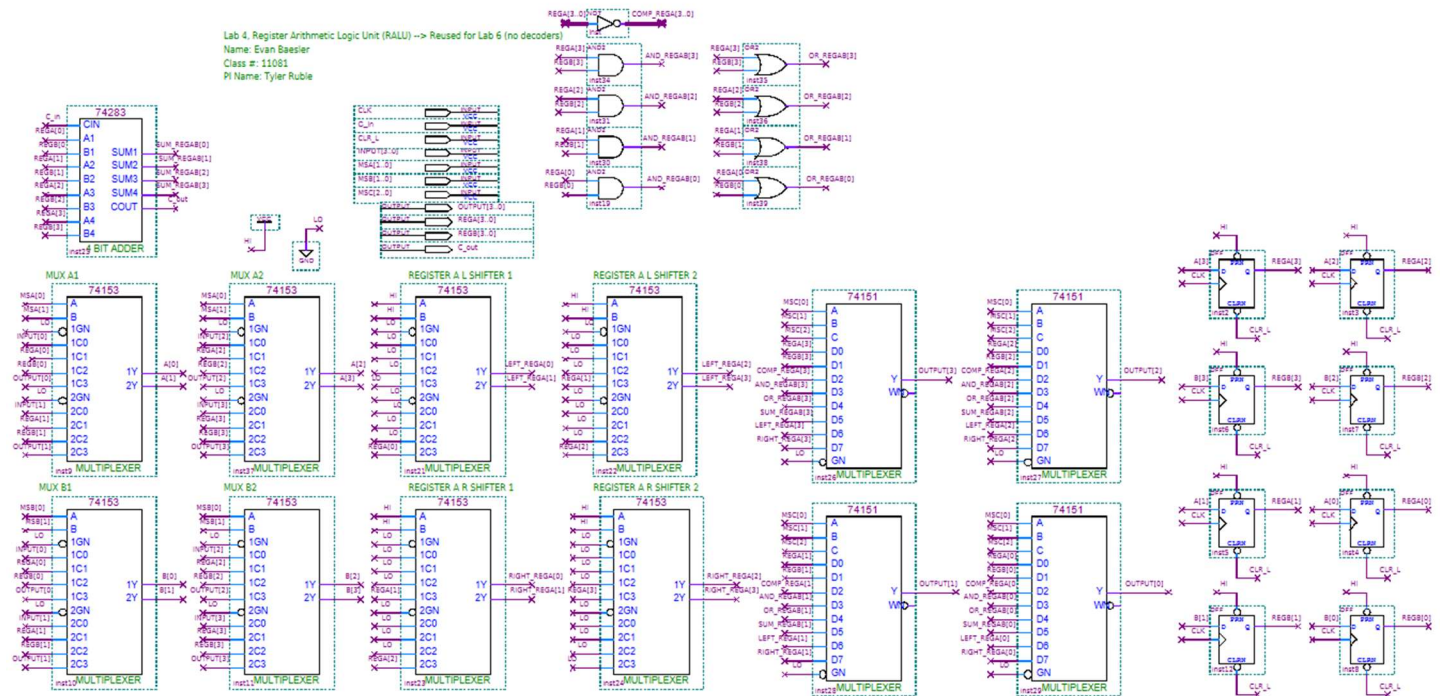


Figure 2: Lab 6 RALU BDF (reused Lab 4)

```
1  library ieee; use ieee.std_logic_1164.all;
2
3  entity Lab6_CONT_COMP is port(
4
5      -- Evan Baesler Part 1 Controller VHDL for Lab 6 - Elementary CPU Design
6
7      -- Instruction Register Variables
8      IR: in std_logic_vector(1 downto 0);
9      IR_LOAD: out std_logic;
10
11     -- Mux Function Select Variables
12     MSA: out std_logic_vector(1 downto 0);
13     MSB: out std_logic_vector(1 downto 0);
14     MSC: out std_logic_vector(2 downto 0);
15
16     -- Q/D for IR, oscillates on clock.
17     Q: in std_logic;
18     D: out std_logic
19 );
20
21 end Lab6_CONT_COMP;
22
23 architecture behavior of Lab6_CONT_COMP is
24
25 begin
26
27     D <= not Q;
28     IR_LOAD <= not Q;
29
30     MSA(1) <= (
31         (Q and not IR(1) and not IR(0)) or
32         (Q and IR(1) and not IR(0))
33     );
34
35     MSA(0) <= (
36         (not Q) or
37         (Q and not IR(1) and not IR(0)) or
38         (Q and not IR(1) and IR(0)) or
39         (Q and IR(1) and not IR(0))
40     );
41
42     -- Always true, keeping expanded incase of future
43     -- additions to IR.
44     MSB(1) <= (
45         (not Q) or
46         (Q and not IR(1) and not IR(0)) or
47         (Q and not IR(1) and IR(0)) or
48         (Q and IR(1) and not IR(0)) or
49         (Q and IR(1) and IR(0))
50     );
51
52     MSB(0) <= (
53         (Q and not IR(1) and IR(0))
54     );
55
56     MSC(2) <= (
57         (Q and not IR(1) and not IR(0)) or
58         (Q and IR(1) and not IR(0))
59     );
60
61     MSC(1) <= (
62         (Q and not IR(1) and not IR(0))
63     );
64
65     MSC(0) <= (
66         (Q and IR(1) and not IR(0))
67     );
68
69 end behavior;
```

Figure 3: Lab 6 Part 1 Controller VHDL

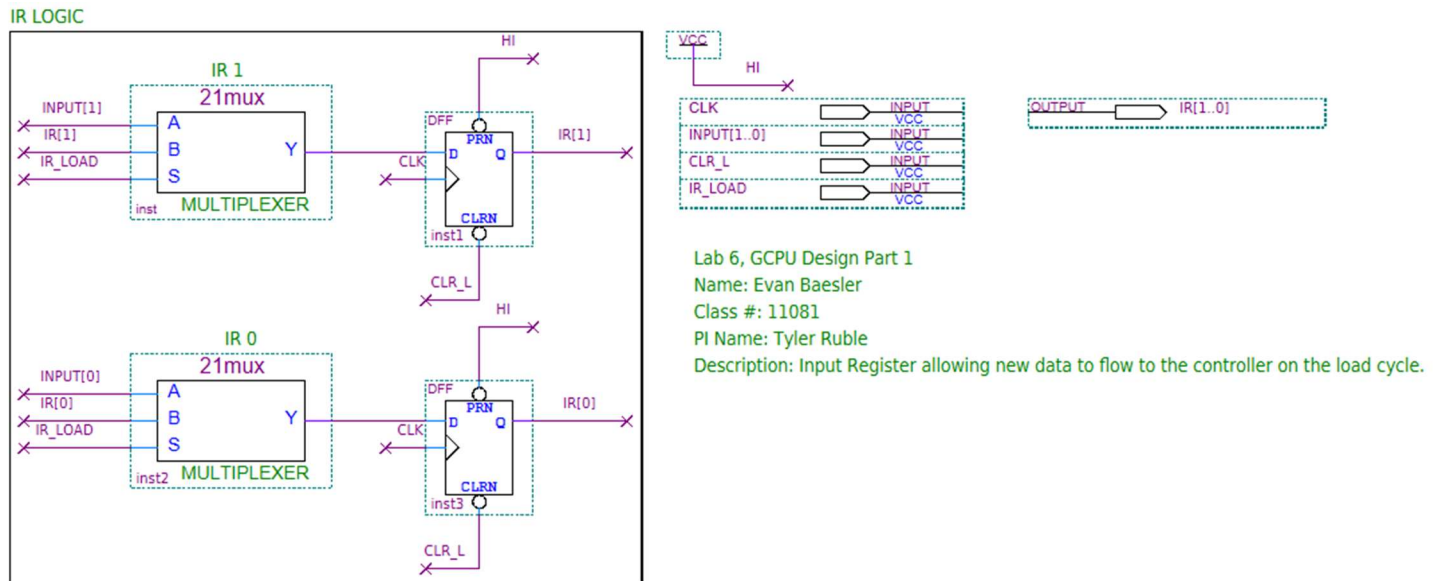


Figure 4: Quartus BDF for Input Register

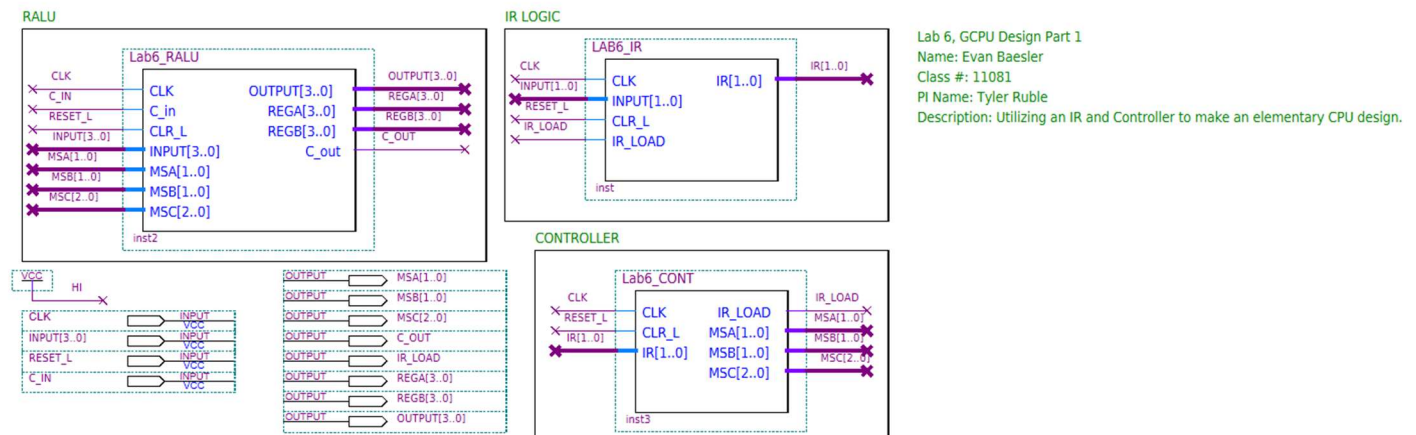


Figure 5: Quartus BDF Schematic for Lab 6 Part 1

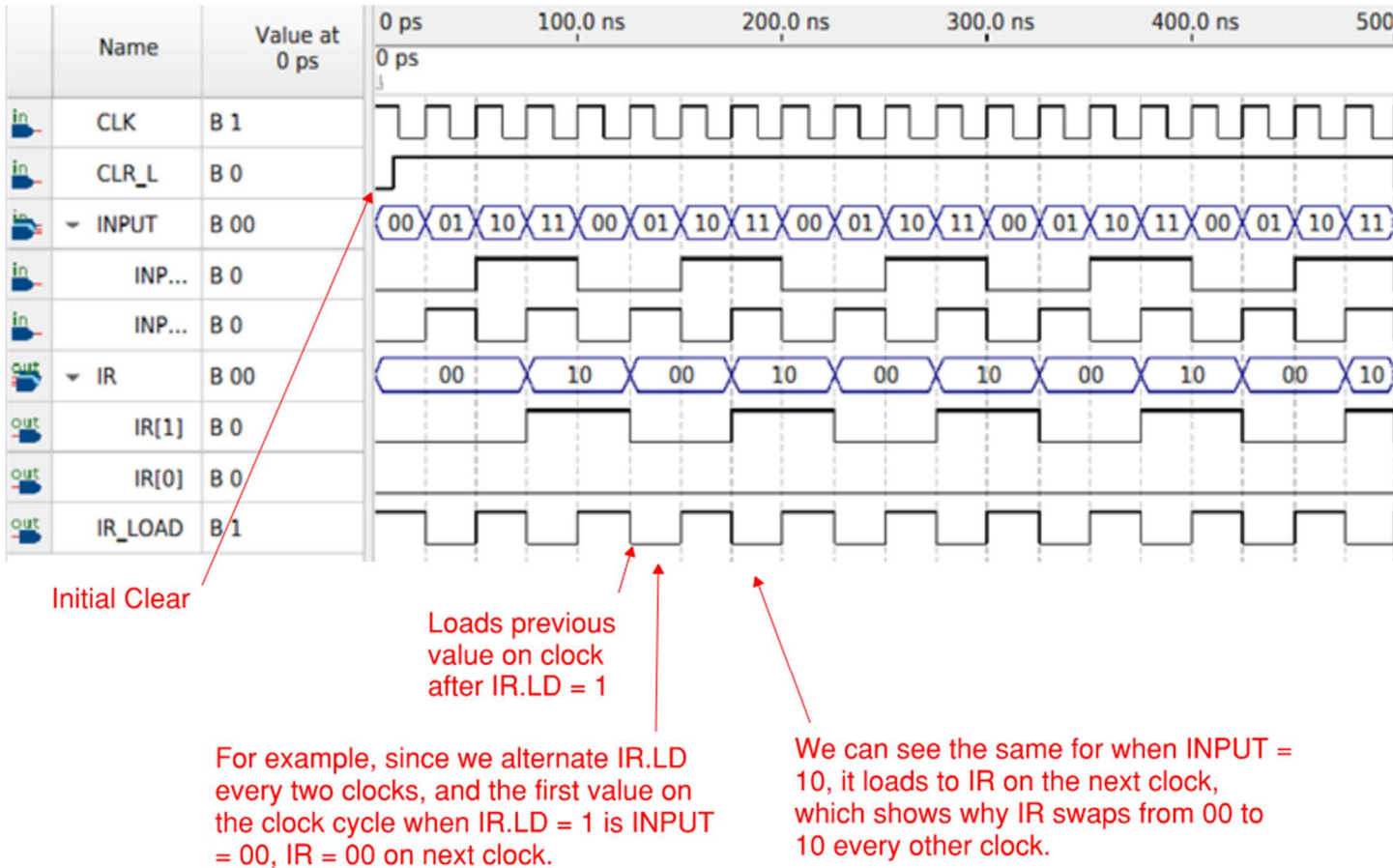


Figure 6: IR.LD Quartus Simulation, showing values being loaded when IR_LOAD is high.

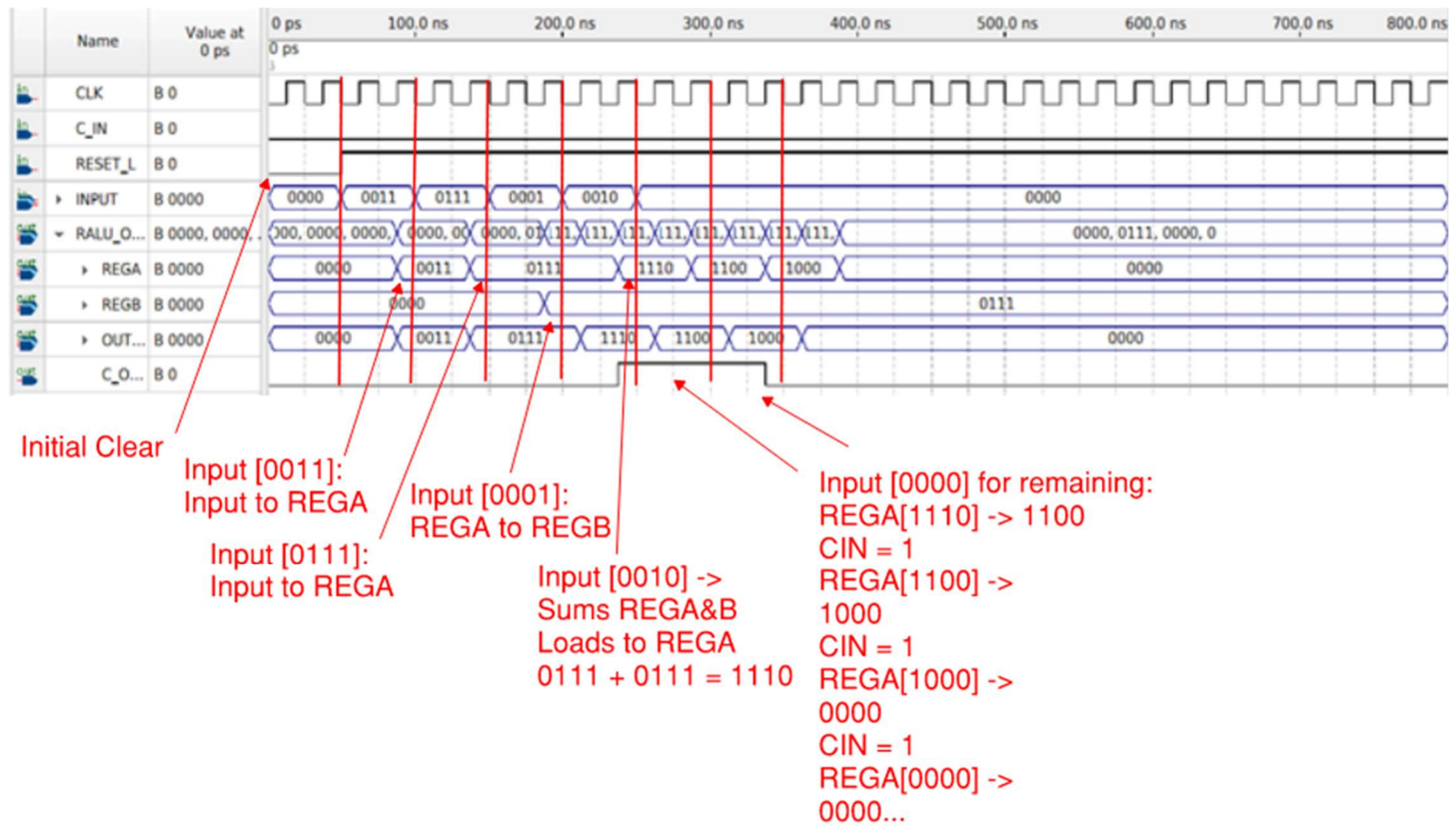


Figure 7: Lab 6 Part 1 - Quartus Waveform Simulation

PRE-LAB PART 1 QUESTIONS

1. Why did we require the new instruction register in this design?
 - a. We use the instruction register to control the input of commands so that they do not interrupt another command before it can be fully executed.
2. In this section of the lab, you are setting the INPUT bus by hand. If you wanted to read or fetch this value from memory, what could you add to do this automatically for you every CLK cycle?
 - a. We would use a program counter and a ROM as done in later in Part 2, which would fetch values from a range of ROM registers and run those through our controller. The program counter iterates while the ROM stores and pushes data.
3. How could you add more instructions (i.e., 8 instead of 4) to the controller?
 - a. By increasing the amount of bits to our input register (4 -> 8, 2 bits -> 3 bits), which would give us more combinations to give to the controller, which gives us more controls.

[illegible]

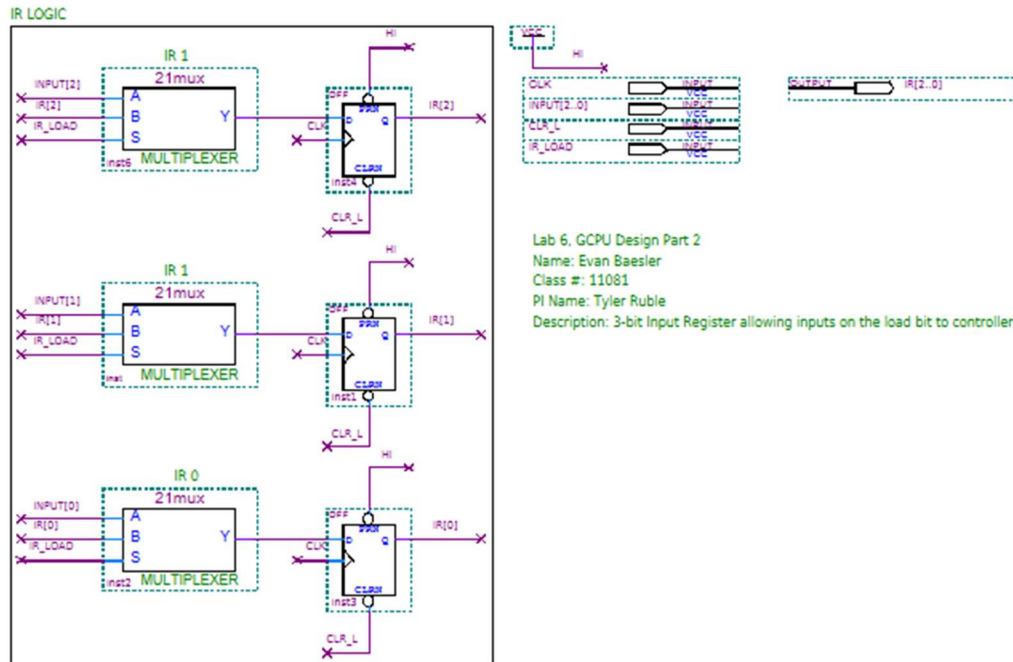


Figure 9: Lab 6 Part 2 3-bit Input Register BDF

```

1  library ieee; use ieee.std_logic_1164.all;
2
3  entity Lab6_CONT_COMP is port(
4
5      -- Lab 6 Part 2 Controller designed by Evan Baessler
6
7      -- Instruction Register Variables
8      IR: in std_logic_vector(2 downto 0);
9      IR_LOAD: out std_logic;
10
11      -- Mux Function Select Variables
12      MSA: out std_logic_vector(1 downto 0);
13      MSB: out std_logic_vector(1 downto 0);
14      MSC: out std_logic_vector(2 downto 0);
15
16      -- Program Counter
17      PC_INC: out std_logic;
18      PC_LD: out std_logic;
19
20      -- Q/D for IR, oscillates on clock.
21      Q: in std_logic_vector(1 downto 0);
22      D: out std_logic_vector(1 downto 0)
23  );
24
25  end Lab6_CONT_COMP;
26

```

Figure 10: Lab 6 Part 2 Controller VHDL Part 1

```

26 architecture behavior of Lab6_CONT_COMP is
27
28     signal PC_LD_H : std_logic;
29
30
31 begin
32
33     PC_LD <= not(PC_LD_H);
34
35     IR_LOAD <= (
36         (not Q(1) and not Q(0))
37     );
38
39     D(1) <= (
40         (not IR(2) and IR(1) and not Q(1) and Q(0))
41     );
42
43     D(0) <= (
44         (not Q(1) and not Q(0)) or
45         (not IR(2) and IR(1) and IR(0) and not Q(1) and Q(0))
46     );
47
48     MSA(1) <= (
49         (not Q(1) and Q(0) and not IR(2) and not IR(1) and not IR(0)) or
50         (not Q(1) and Q(0) and IR(2) and not IR(1))
51     );
52
53     MSA(0) <= (
54         (not Q(1) and not Q(0)) or
55         (not Q(1) and Q(0) and not IR(2) and not IR(1) and not IR(0)) or
56         (not Q(1) and Q(0) and not IR(2) and not IR(1) and IR(0)) or
57         (not Q(1) and Q(0) and not IR(2) and IR(1) and not IR(0)) or
58         (not Q(1) and Q(0) and not IR(2) and IR(1) and IR(0)) or
59         (not Q(1) and Q(0) and IR(2) and not IR(1) and not IR(0)) or
60         (not Q(1) and Q(0) and IR(2) and not IR(1) and IR(0)) or
61         (Q(1) and Q(0))
62     );
63
64     MSB(1) <= (
65         (not Q(1) and not Q(0)) or
66         (not Q(1) and Q(0) and not IR(2) and not IR(1) and not IR(0)) or
67         (not Q(1) and Q(0) and not IR(2) and not IR(1) and IR(0)) or
68         (not Q(1) and Q(0) and not IR(2) and IR(1) and not IR(0)) or
69         (Q(1) and not Q(0)) or
70         (not Q(1) and Q(0) and not IR(2) and IR(1) and IR(0)) or
71         (Q(1) and Q(0)) or
72         (not Q(1) and Q(0) and IR(2) and not IR(1))
73     );
74
75     MSB(0) <= (
76         (not Q(1) and Q(0) and not IR(2) and not IR(1) and IR(0))
77     );
78
79     MSC(2) <= (
80         (not Q(1) and Q(0) and not IR(2) and not IR(1) and not IR(0)) or
81         (not Q(1) and Q(0) and IR(2) and not IR(1))
82     );
83
84     MSC(1) <= (
85         (not Q(1) and Q(0) and IR(2) and not IR(1))
86     );
87
88     MSC(0) <= (
89         (not Q(1) and Q(0) and not IR(2) and not IR(1) and not IR(0)) or
90         (not Q(1) and Q(0) and IR(2) and not IR(1) and not IR(0))
91     );
92
93     PC_LD_H <= (
94         (Q(1) and Q(0))
95     );
96
97     PC_INC <= (
98         (not Q(1) and Q(0)) or
99         (Q(1) and not Q(0))
100     );
101
102 end behavior;

```

Figure 11: Lab 6 Part 2 Controller VHDL Part 2

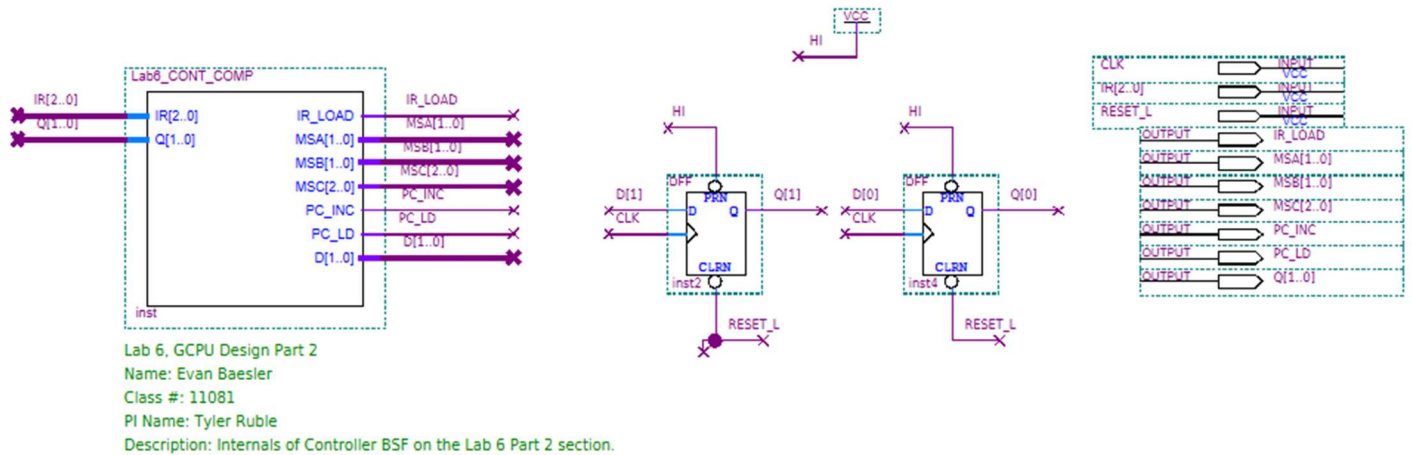


Figure 12: Lab 6 Part 2 Controller Internals BDF

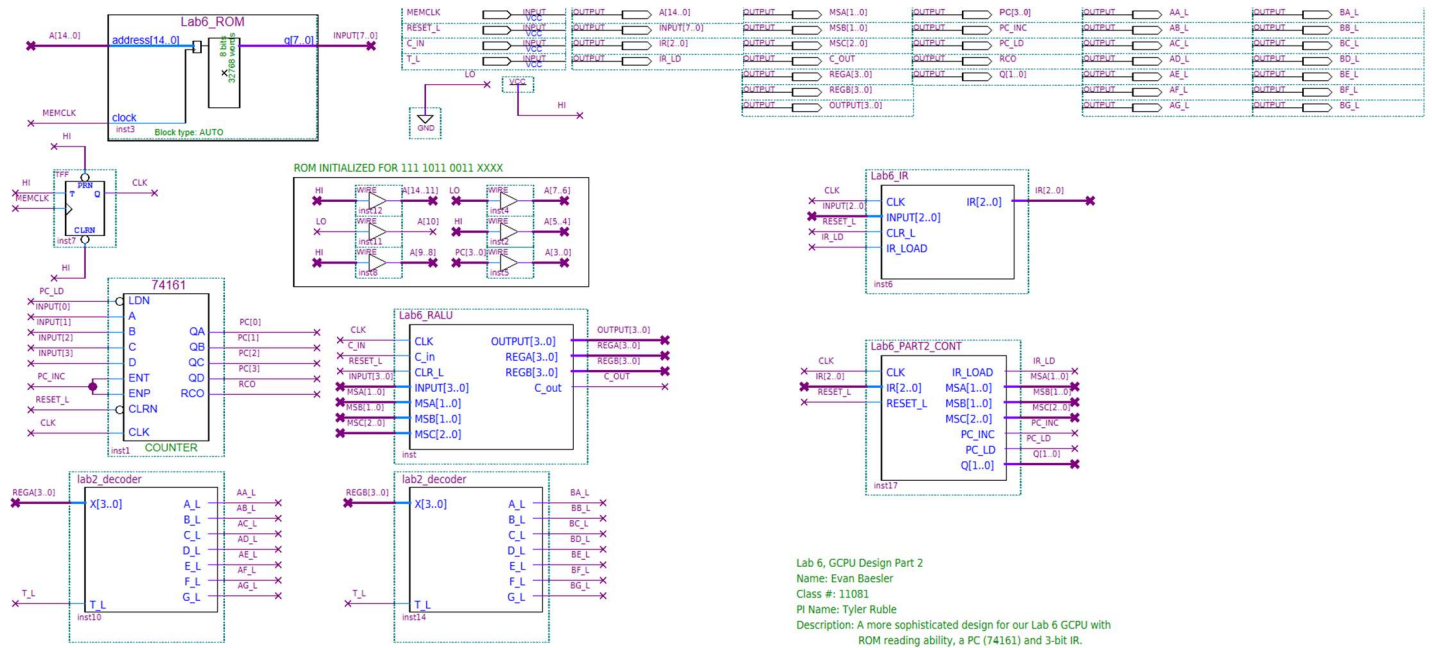


Figure 13: Lab 6 Part 2 CPU BDF utilizing 74161 Program Counter and Quartus .mif ROM

Table 3: Programming Table for ROM .mif

Addr.		Mach Codes	A	B	A	B	A	B	A	B
\$7B30	LDAA #3	02, 03	3	0						
\$7B32	TAB	01	3	3	5	5	2	2	3	3
\$7B33	LDAA #7	02, 07	7	3	7	5	7	2	7	3
\$7B35	ABA	00	A	3	C	5	9	2	A	3
\$7B36	SAL	05	4	3	8	5	2	2	4	3
\$7B37	SAL	05	8	3	0	5	4	2	8	3
\$7B38	ABA	00	B	3	5	5	6	2	B	3
\$7B39	SAR	04	5	3	2	5	3	2	5	3
\$7B3A	JMP #2	03, 02	5	3	2	5	3	2	5	3
\$7B3C	LDAA #\$B	03, 11								
\$7B3E	ABA	00								
\$...										

```
-- File name= rom_32k.mif

DEPTH = 32768; % Memory depth and width are required %
WIDTH = 8; % Enter a decimal number %

ADDRESS_RADIX = HEX; % Address and value radices are optional %
DATA_RADIX = HEX; % Enter BIN, DEC, HEX, or OCT; unless %
% otherwise specified, radices = HEX %

-- Specify values for addresses, which can be single address or range

CONTENT
BEGIN

[0..7B2F] : 0; % First values thru 7B2F are zero %
7B30 : 2; % Single address data %
7B31 : 3;
7B32 : 1;
7B33 : 2;
7B34 : 7;
7B35 : 0;
7B36 : 5;
7B37 : 5;
7B38 : 0;
7B39 : 4;
7B3A : 3;
7B3B : 2;
7B3C : 3;
7B3D : 11;
[7B3E..7FFF] : 00;
END ;
```

Figure 14: Code from .mif File

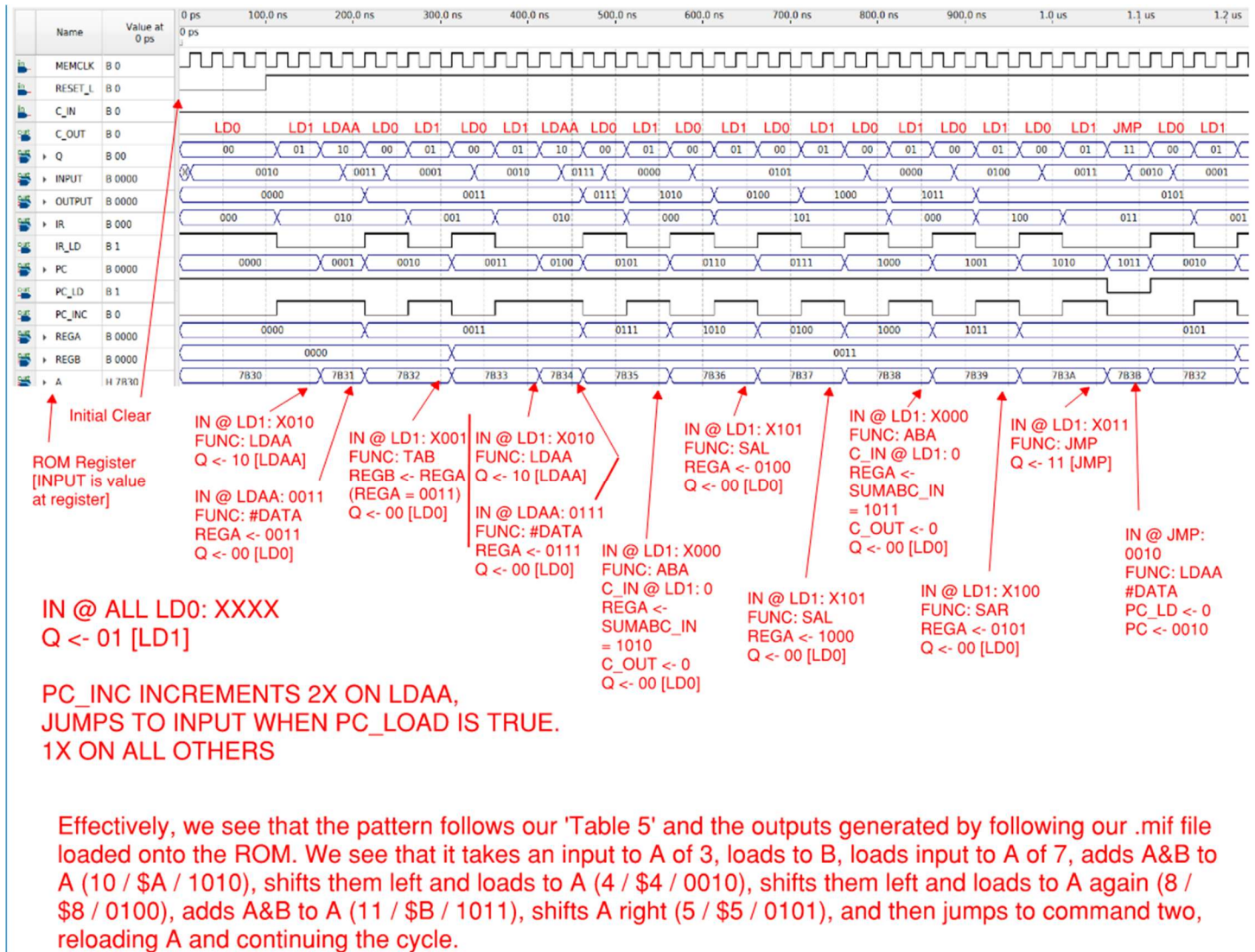


Figure 15: Annotated Quartus Waveform Simulation for Lab 6 Part 2

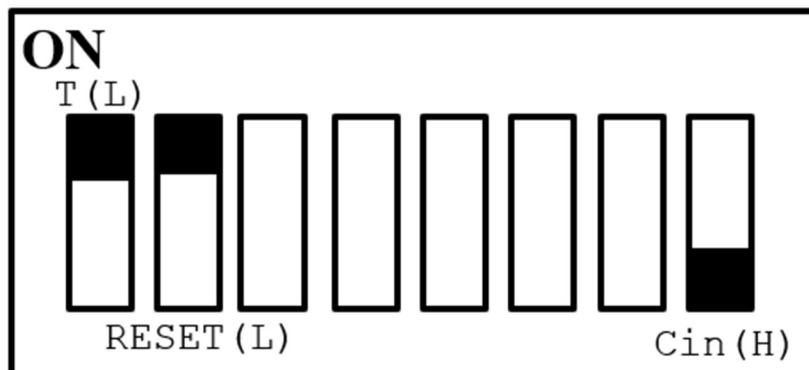


Figure 16: Switch Diagram for Lab 6 Part 2

PRE-LAB PART 2 QUESTIONS

1. Why do we need the extra states in the LDAA and JMP instruction paths?
 - a. One state loads the function, the next loads the data for that function. For LDAA, we first tell the system we are going to load A with a value, then we choose a value to load A with. For JMP, we tell the system we are going to jump, and then we tell it what command to jump to.
2. What do you need to do to the address lines to get your program to start at the address \$16E0 (instead of \$7B30)?
 - a. For \$7B30, we initialized our ROM by converting the hex to binary -> 111 1011 0111 XXXX, where XXXX is set to 0 via reset. To change this we do the same method, which results in 001 0110 1110 XXXX. We set the defined bits to VCC and GND respectively, and set X's to Reset_L.